

Relatório de Auditoria de Segurança

INOVAPRO3D · loja e calculadora de impressão 3D

1 de setembro de 2026 · branch `main` · commit `e099126`

ALTA 1 MÉDIA 2 BAIXA 4 INFO 1

Escopo auditado

Auditoria de caixa-branca do repositório completo: **frontend** (216 arquivos TypeScript/TSX em `src/`), **backend** (11 funções serverless em `api/`, 12 rotas Express em `server.ts`, 20 módulos em `server/`), **regras de autorização** (`firestore.rules`, `storage.rules`), **configuração de deploy** (`vercel.json`, `firebase.json`, GitHub Actions), **scripts de build** e **todo o histórico do git**. Não houve teste dinâmico contra ambiente em execução: as conclusões vêm da leitura do código no commit indicado.

Stack detectada

Linguagem	TypeScript (ESM), Node.js 22
Frontend	React 19 + Vite 6 + React Router 7 + Tailwind 4
Backend	Express 4 (self-host, <code>server.ts</code>) e Vercel Serverless Functions (<code>api/</code>) — a Vercel é o runtime de produção
Persistência	Cloud Firestore acessado direto do navegador (SDK web) + Firebase Admin SDK no servidor
Autorização	Firestore Security Rules (<code>firestore.rules</code>) + Storage Rules (<code>storage.rules</code>) — não há ORM nem camada de query no servidor
Autenticação	Firebase Auth (Google Sign-In); papel em <code>users/{uid}.role</code> , sem custom claims
Pagamentos	Mercado Pago (Pix) e Stripe
Deploy	Vercel (<code>vercel.json</code>) + GitHub Actions; sem Docker, Helm ou Terraform

Nota metodológica

As cinco categorias do roteiro foram traduzidas para os equivalentes desta stack antes da busca. Como o projeto **não usa ORM nem camada de query no servidor** — o navegador conversa direto com o Firestore —, o mecanismo de isolamento de dados é o arquivo de regras, e não um filtro **WHERE** em código. Isso muda onde cada categoria é procurada:

1. Banco sem tranca

O projeto não usa ORM: o navegador fala direto com o Firestore, então o **mecanismo de isolamento é o arquivo de regras**. A categoria foi mapeada para uma leitura linha a linha de **firestore.rules** (367 linhas) e **storage.rules** (55 linhas), cruzando cada *match* com as coleções realmente usadas no código.

2. Permissão definida no navegador

Cada gate de papel do frontend (**ProtectedRoute requireAdmin, profile?.role === "ADMIN"**) foi cruzado com o ponto que executa a operação. Como as escritas administrativas vão direto ao Firestore, o equivalente server-side é a função **isAdmin()** das regras — verificada coleção por coleção.

3. IDOR

Percorridos **todos** os 16 handlers de rota do backend (11 funções serverless em **api/** e 12 rotas Express em **server.ts**), conferindo, em cada um que recebe um identificador de objeto, se há checagem de posse antes de ler ou escrever.

4. Chaves expostas

Varredura de código, configs, **vercel.json**, workflows do GitHub Actions, scripts, documentação e **todo o histórico do git** (todos os commits, todas as refs). Também foi inspecionado o bundle compilado em **dist/assets/** à procura de segredos de servidor que tivessem vazado para o navegador.

5. Inputs sem tratamento (XSS)

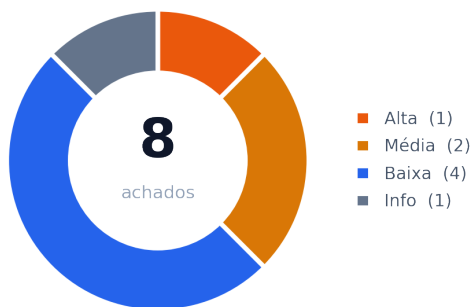
Busca por **dangerouslySetInnerHTML, innerHTML, outerHTML, insertAdjacentHTML, eval, new Function** e **document.write** no frontend; e, no servidor, rastreio de todo dado de origem externa que entra em HTML de e-mail ou em mensagem do Telegram (ambos os canais interpretam markup).

Categorias sem aplicação nesta stack

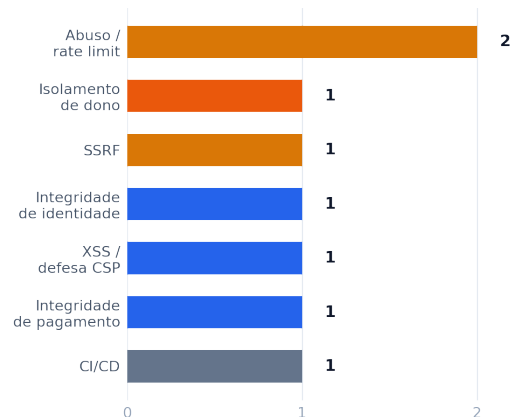
Injeção de SQL e isolamento multi-tenant por organização não se aplicam: não há banco relacional nem *query builder*, e o modelo de dados é de dono individual (**userId**), sem conceito de organização, workspace ou tenant. **Segredos em Docker, Helm ou Terraform** não se aplicam: o deploy é serverless na Vercel e não há nenhum desses arquivos no repositório. **Sanitização de HTML no frontend** foi verificada e não se aplica na prática — não existe ponto de renderização de HTML bruto onde uma biblioteca de sanitização seria instalada (detalhado nos Pontos Fortes).

Resumo executivo

Foram identificados **8 achados**: **nenhum crítico**, **1 de severidade alta**, **2 médios**, **4 baixos** e **1 informativo**. O projeto chega a esta auditoria com um trabalho de endurecimento já feito e visível: as regras do Firestore fecham por padrão e são testadas automaticamente, o preço do pedido é recalculado no servidor, todo handler que recebe um ID de pedido confere a posse, e não há um único ponto de injeção de HTML no frontend. Os achados restantes concentram-se em três frentes: um anexo que não herdou a proteção do seu documento, defesas presentes no runtime Express que não foram portadas para o runtime serverless de produção, e camadas de contenção construídas mas ainda não acionadas.



Distribuição por severidade



Achados por categoria (cor = maior severidade da categoria)

Achados em uma tabela

SEV.	ID	ARQUIVO : LINHA	DESCRIÇÃO
ALTA	A1	storage.rules:33-36	Anexos de orçamento com leitura pública no Firebase Storage
MÉDIA	A2	api/report-error.ts:20-46	Endpoint anônimo de produção grava no Firestore sem nenhum rate limit
MÉDIA	A3	server.ts:752-762 _modelMetadata.ts:260-285	Allowlist de host validada só na URL inicial; redirects seguidos sem revalidar
BAIXA	A4	server.ts:65-83 csp-report.ts:21 payment-status.ts:8 process-payment.ts:11	Rate limiter guardado em memória do processo é inefetivo em runtime serverless
BAIXA	A5	firestore.rules: 127-136, 237-241	Nome e foto exibidos em avaliações não são amarrados ao usuário autenticado
BAIXA	A6	vercel.json:40-41 server.ts:837	CSP publicada apenas em Report-Only: a política existe mas não bloqueia
BAIXA	A7	server.ts:249-263	Webhook do Stripe marca pedido como PAGO sem conferir valor nem estado anterior
INFO	A8	.github/workflows/ci.yml:1-9	Workflow ci.yml não declara permissions do GITHUB_TOKEN

Pontos fortes

Esta seção existe para provar a cobertura da auditoria: registra o que foi efetivamente verificado e está correto, com a evidência no código. Um achado ausente aqui é um lugar onde não se olhou; tudo abaixo foi olhado.

✓ Firestore fecha por padrão e abre coleção por coleção

`firestore.rules:157-159`

`match /{document=**} { allow read, write: if false; }` abre o arquivo, e cada uma das 23 coleções recebe regra explícita. Nenhuma coleção usada no código ficou sem regra — e as coleções escritas só pelo Admin SDK (`paymentAttempts`, `paymentEvents`, `cspReports`) deliberadamente não têm regra, caindo na negação padrão.

✓ Isolamento por dono aplicado nas coleções sensíveis

`firestore.rules:290-295, 354-358, 361-365`

`orders` e `quotes` permitem leitura apenas ao dono ou ao admin; `savedCalculations` separa as operações (`read/delete`, `create`, `update`) para impedir a criação de documento com `userId` de terceiro, e o `update` exige que o dono não mude. As consultas do cliente filtram de acordo — `src/pages/public/MyOrders.tsx:37` usa `where("userId", "==", user.uid)`.

✓ Auto-promoção a administrador é impossível

`firestore.rules:53-60, 162-167`

`isValidProfileSelfUpdate()` restringe a atualização do próprio perfil a `hasOnly(['name','firstName','lastName','phone','addresses','photoURL'])` — `role` está fora. A criação força `role == 'CUSTOMER'` e `email == authEmail()`, e `users` não tem `allow delete`, então não dá para apagar e recriar o documento com outro papel. `list` também não é concedido: a coleção de usuários não é enumerável.

✓ Nenhum IDOR: todos os handlers que recebem ID verificam posse

`server.ts:202, 652` · `api/mercadopago/payment-status.ts:92` · `server/_orderNotification.ts:107` · `server/mercadopago/_service.ts:96`

Os 16 handlers foram percorridos individualmente. Os cinco que recebem um identificador de pedido comparam `order.userId` com o UID do token verificado antes de qualquer leitura ou escrita: criação de `PaymentIntent`, consulta de status (nas duas implementações), notificação de pedido e processamento de Pix. Os demais não operam sobre objeto identificado por ID.

✓ Preço recalculado no servidor a partir do catálogo

`api/orders/create.ts:79-126` · `server.ts:392-424`

O cliente envia apenas itens e quantidades; `computeOrderTotal` recarrega os preços de `products` pelo Admin SDK. `userName` e `userEmail` do corpo são ignorados de propósito e substituídos por `resolveTrustedIdentity()`, que só aceita e-mail com claim `email_verified` (`server/_orderNotification.ts:35-38`). Fecha manipulação de preço via `localStorage` e o uso da rota como relay de e-mail.

✓ Verificação de token falha fechada

`server.ts:107-116` · `api/orders/create.ts:39-48`

`verifyToken` devolve `null` em qualquer falha, e as rotas recusam quando o Admin SDK não está configurado (`503`) em vez de degradar para modo sem autenticação. O comentário no código registra que a versão anterior devolvia a string `"unchecked"` — que passava no teste `if (!uid)` — e que isso foi corrigido.

✓ Webhook de pagamento com assinatura HMAC e conferência de valor

`server/mercadopago/_webhook.ts:46-75` · `server/mercadopago/_webhookService.ts:48,99-111`

A assinatura usa `timingSafeEqual`, valida o formato do hash, e rejeita timestamp com mais de 5 minutos (anti-replay). O payload recebido nunca é fonte de verdade: o pagamento é **reconsultado na API do provedor**, o valor é comparado com o total do pedido (`amount_mismatch`) e a transição de estado acontece dentro de uma transação com registro em `paymentEvents`. A criação de Pix também é transacional e idempotente, evitando cobrança dupla em clique repetido.

✓ Frontend sem nenhum vetor de injeção de HTML

varredura de `src/` (216 arquivos `.ts/.tsx`) e `index.html`

Zero ocorrências de `dangerouslySetInnerHTML`, `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `eval`, `new Function` e `document.write`. Todo conteúdo gerado por usuário — comentários de avaliação, nomes, notas — é renderizado como texto JSX, que o React escapa. Não há renderização de Markdown nem de HTML recebido. Por isso o projeto não precisa de biblioteca de sanitização: não há ponto onde ela seria aplicada.

✓ Escape centralizado nos canais que interpretam markup

`server/_escapeHtml.ts` · `_emailTemplates.ts:40,44,54,78-82` · `_orderNotification.ts:157-164` · `_reportError.ts:59-64`

E-mail e Telegram renderizam HTML. Todo valor interpolado passa por `escapeHtml()` — inclusive os que vêm do Firestore, que um dia foram digitados por alguém. O módulo é puro e compartilhado pelos dois runtimes, com testes próprios (`server/_escapeHtml.test.ts`, `_emailTemplates.test.ts`).

✓ Nenhum segredo no código, nas configs ou no histórico do git

`.gitignore:7-8` · `.env.example` · varredura de todos os commits

`.env.example` traz somente chaves vazias e comentários. A varredura de **todos** os commits de **todas** as refs não encontrou chave privada, token de bot, segredo de webhook ou credencial de API. Nenhum arquivo `.env` real foi rastreado. Não há defaults inseguros do tipo `${VAR:-segredo}` — os únicos fallbacks literais são a URL pública do site e o nome do remetente. Os segredos de servidor ficam em `server/mercadopago/_config.ts`, deliberadamente fora de `src/`.

✓ Bundle do navegador carrega apenas chaves públicas por natureza

`dist/assets/*.js` · `src/lib/mercadopago/config.ts:1-7`

A inspeção do bundle compilado encontrou a Firebase Web API Key e a Public Key do Mercado Pago (`APP_USR-<uuid>`, formato de chave pública, não de access token). As duas são identificadores públicos por desenho. Nenhum `STRIPE_SECRET_KEY`, `MERCADOPAGO_ACCESS_TOKEN`, `FIREBASE_PRIVATE_KEY`, `SENDPULSE_API_SECRET`, `GEMINI_API_KEY` ou token do Telegram vazou para o navegador.

✓ Regras de segurança cobertas por testes automatizados

`tests/rules/firestore.rules.test.ts` (275 linhas) · `package.json:test:rules`

Suíte executada contra o emulador do Firestore cobrindo negação de leitura de **coupons** por cliente, ID determinístico de **reviews/reviewVotes/reviewReports**, tentativa de gravar no ID de outro usuário, inscrição de newsletter e campos opcionais ausentes. Regra de segurança neste projeto é código testado, não comentário.

Pontos fracos

Os riscos centrais, na ordem em que importam. Cada um está detalhado adiante com arquivo, linha e trecho de código.

1. O anexo não herda a tranca do documento

O risco central. O orçamento está protegido no Firestore, mas a imagem que o acompanha está aberta no Storage — e **read** ali também concede **list**, o que torna o acervo enumerável, não só acessível por URL adivinhada. É o único achado que expõe dado de cliente sem depender de nenhuma outra condição.

2. As defesas do Express não foram todas portadas para a Vercel

O projeto mantém dois runtimes com a mesma superfície de API, e a Vercel é a de produção. O rate limit de **/api/report-error** existe só no Express; o webhook do Stripe existe só no Express. Sempre que os dois divergem, é a produção que fica descoberta — e a divergência não é visível em revisão de código, porque cada arquivo isolado parece correto.

3. Allowlist de host que não sobrevive ao primeiro redirect

Duas rotas anônimas validam o destino e depois deixam o **fetch** seguir para onde o servidor remoto mandar. A allowlist inclui sites de conteúdo de usuário, onde open redirect é achado comum.

4. Contadores de abuso presos à memória do processo

Todo limitador do projeto guarda estado num **Map** local. Sob escala horizontal, o teto configurado deixa de ser o teto real — o que transforma cada endpoint anônimo numa superfície de custo.

5. A camada que conteria um XSS está desligada

O frontend hoje não tem vetor de injeção, e a CSP está bem escrita — mas publicada em Report-Only. É proteção construída e não acionada: o dia em que uma regressão introduzir um **dangerouslySetInnerHTML**, nada a impedirá.

Achados detalhados

Cada ficha traz a categoria, a localização exata, o trecho de código, por que é explorável, o impacto e a correção sugerida. Nenhum achado abaixo é especulativo: todos foram lidos no código no commit auditado.

1. Banco sem tranca (isolamento de dono)

ALTA

A1 · Anexos de orçamento com leitura pública no Firebase Storage

[storage.rules:33-36](#)

```
// Imagens opcionais anexadas a orçamentos salvos pela calculadora.
match /quotes/{allPaths=**} {
  allow read: if true; // ← qualquer pessoa, sem login
  allow write: if isAdmin() && isImage() && request.resource.size < 10 * 1024 * 1024;
}
```

O que é. O documento do orçamento no Firestore é corretamente restrito ao dono ou ao admin ([firestore.rules:361-365](#)), mas o **anexo correspondente no Storage é público**. O upload grava em `quotes/{uid}/{timestamp}-{nome}-{ext}` ([src/lib/quotes.ts:223](#)), então o caminho carrega o UID do cliente e o nome original do arquivo.

Por que é explorável. Nas Storage Rules, **read** concede **get** e **list**. Com leitura liberada, um visitante não autenticado pode **enumerar o prefixo quotes/ com listAll()** e baixar todas as imagens — sem precisar adivinhar URL. Não depende de feature flag nem de configuração insegura: é o estado publicado da regra.

Impacto. Vazamento do portfólio de peças e projetos dos clientes, dos nomes de arquivo enviados e da correlação UID → peças. Para uma operação B2B de impressão 3D, isso expõe trabalho de terceiros sob sigilo comercial.

Correção. Trocar **allow read: if true** por **allow read: if isAdmin()** nesse bloco. Se a imagem precisar ser exibida ao cliente dono, restringir ao UID do próprio caminho (**request.auth.uid == uid** capturando o segmento) em vez de liberar o prefixo inteiro. As demais pastas (`products`, `showcase`, `categories`, `printers`, `company`) são conteúdo de vitrine e podem seguir públicas.

4. Superfície de abuso / configuração de deploy

MÉDIA

A2 · Endpoint anônimo de produção grava no Firestore sem nenhum rate limit

[api/report-error.ts:20-46](#)

```
export default async function handler(req: VercelRequest, res: VercelResponse) {
  if (req.method !== "POST") { /* ... */ }
  try {
    const { valid, data, telegramText } = buildErrorReport(req.body || {});
    if (!valid) { /* ... */ }
    let id: string | null = null;
    if (isAdminSdkConfigured()) {
      const ref = await getAdminDb().collection("errorReports").add(data); // grava sem auth
      id = ref.id;
    }
    await notifyTelegram(telegramText + ...); // dispara no canal de operacao
  }
}
```

O que é. A rota é anônima por desenho (erro acontece para visitante deslogado) e escreve no Firestore via Admin SDK, além de disparar mensagem no Telegram. A versão Express da mesma rota é protegida por **rateLimit(20)** ([server.ts:671](#)), mas a **função serverless — que é o runtime de produção na Vercel — não tem limitador algum**.

Por que é explorável. Basta um laço de POSTs com corpo `{"message":"x"}`. Nenhum token, nenhum header especial. Cada requisição cria um documento novo em **errorReports** (via `.add()`, sem deduplicação) e envia uma mensagem ao canal de operação.

Impacto. Inflação ilimitada da coleção **errorReports** (custo de escrita e armazenamento no Firestore), inundação do canal do Telegram até o bot ser limitado pela API, e afogamento de relatos legítimos — o painel de erros do admin fica inutilizável.

Correção. Portar o limitador para a função serverless — o mesmo padrão já usado em `api/csp-report.ts:34-50` — e, para valer sob escala horizontal, ancorá-lo em armazenamento compartilhado (ver A4). Combinar com deduplicação por fingerprint da mensagem, como `_cspReportRecorder.ts` já faz.

3. Confiança em entrada externa (SSRF)

MÉDIA

A3 · Allowlist de host validada só na URL inicial; redirects seguidos sem revalidar

`server.ts:752-762` · `server/_modelMetadata.ts:260-285`

```
// server.ts - /api/proxy-image (sem autenticação)
if (!isAllowedImageHost(parsed.hostname)) { // 752 - valida a URL inicial
  res.status(403).json({ error: "host não permitido" });
  return;
}
const upstream = await fetch(rawUrl, {
  headers: { "User-Agent": "..."},
  redirect: "follow", // 761 - destino final não é revalidado
});

// server/_modelMetadata.ts - readModelMetadata (sem autenticação)
if (!isAllowedImportHost(targetUrl.hostname)) { /* 260 */ }
const response = await fetch(targetUrl, { headers: {...}, redirect: "follow" }); // 284
```

O que é. Os dois pontos checam o host **antes** do fetch e depois seguem redirects automaticamente. O destino final nunca volta à allowlist. A lista padrão (`server.ts:48-54`) inclui sites de conteúdo gerado por usuário — thingiverse.com, printables.com, cults3d.com, myminifactory.com — e o casamento aceita subdomínios (`host.endsWith(".") + allowed`).

Por que é explorável. Condicionada a um **open redirect em qualquer host da allowlist** — classe comum em sites com perfis e links de usuário. Com um, o atacante alcança endereços internos (**169.254.169.254, 10.0.0.0/8, localhost**). Em `/api/proxy-image` o corpo da resposta volta ao chamador quando o **Content-Type** começa com `image/`, cabeçalho que o servidor de destino controla. Ambas as rotas são **anônimas**.

Impacto. Leitura de serviços internos e de endpoints de metadados da nuvem a partir do IP do servidor, com exfiltração do conteúdo pela resposta do proxy. Em `/api/model-metadata`, também uso do servidor como proxy de leitura anônimo.

Correção. Usar `redirect: "manual"` e reaplicar `isAllowedImageHost/isAllowedImportHost` a cada salto, com teto de saltos. Antes de conectar, resolver o hostname e rejeitar IPs privados, loopback e link-local. Restringir a allowlist aos domínios efetivamente necessários e exigir autenticação onde o recurso não é público.

4. Superfície de abuso / configuração de deploy

BAIXA

A4 · Rate limiter guardado em memória do processo é inefetivo em runtime serverless

`server.ts:65-83` · `api/csp-report.ts:21` · `api/mercadopago/payment-status.ts:8` · `api/mercadopago/process-payment.ts:11`

```
// server.ts:65 - e o mesmo padrão repetido nas funções serverless
const rateBuckets = new Map<string, { count: number; resetAt: number }>();
function rateLimit(maxPerMinute: number) {
  return (req, res, next) => {
    const key = `${req.path}:${req.ip}`;
    // ... contador vive apenas nesta instância do processo
```

O que é. Os contadores ficam num **Map** local. Em Express de instância única isso funciona; na Vercel, cada invocação pode cair numa instância diferente e instâncias frias começam com o mapa vazio.

Por que é explorável. Não exige truque: o próprio modelo de escala do provedor dilui o limite. O teto efetivo vira *máximo por minuto × número de instâncias ativas*, valor que o atacante aumenta simplesmente paralelizando as requisições.

Impacto. Enfraquece a proteção de `/api/mercadopago/process-payment` (chamadas ao provedor de pagamento), `/api/csp-report` e `/api/calculator/extract-slicer` (cota da API Gemini). É o multiplicador que agrava A2.

Correção. Mover a contagem para armazenamento compartilhado (Vercel KV, Upstash Redis ou um documento do Firestore com **FieldValue.increment**) ou adotar o rate limiting da borda do provedor. Manter o limitador em memória apenas como segunda linha.

1. Banco sem tranca (integridade de identidade)

BAIXA

A5 · Nome e foto exibidos em avaliações não são amarrados ao usuário autenticado

firestore.rules:127-136, 237-241

```
function isValidReview(data) {
  return data.productId is string && data.productId.size() <= 128 &&
    data.userId is string &&
    data.rating is number && data.rating >= 1 && data.rating <= 5 &&
    optionalString(data.get('comment', null), 1000) &&
    optionalString(data.get('userName', null), 160) && // ← conteúdo livre
    optionalString(data.get('userPhoto', null), 2048) && // ← URL livre
    /* ... */;
}
allow create: if isSignedIn()
  && isValidReview(incoming())
  && incoming().userId == request.auth.uid // ← userId é amarrado
  && reviewId == incoming().productId + '_' + request.auth.uid
  && !('hidden' in incoming());
```

O que é. A regra amarra corretamente `userId` e força um ID determinístico (uma avaliação por usuário por produto), mas `userName` e `userPhoto` — os campos efetivamente exibidos — aceitam qualquer string. O cliente legítimo envia os dados do próprio perfil (`src/hooks/useReviews.ts:91-92`), porém isso é convenção do frontend, não regra.

Por que é explorável. Qualquer usuário autenticado escreve o documento direto pelo SDK web com `userName`: "INOVAPRO3D Oficial" e um `userPhoto` apontando para qualquer URL. A avaliação é renderizada publicamente em `src/components/product/ProductReviews.tsx:211,223`.

Impacto. Personificação da marca ou de outro cliente em avaliações públicas de produto, com peso de prova social. A URL da foto também vira um beacon: carrega do navegador de todo visitante da página do produto, revelando IP e User-Agent a um terceiro escolhido pelo autor da avaliação.

Correção. Ou remover `userName/userPhoto` do documento e resolvê-los na leitura a partir de `users/{userId}`, ou exigir na regra que casem com o token (`incoming().userName == request.auth.token.name`). Para a foto, restringir o host a domínios conhecidos de avatar.

5. Inputs sem tratamento (XSS)

BAIXA

A6 · CSP publicada apenas em Report-Only: a política existe mas não bloqueia

vercel.json:40-41 · server.ts:837

```
// vercel.json:40
{ "key": "Content-Security-Policy-Report-Only", "value": "default-src 'self'; script-src 'self' 'sha256-...' ..." }

// server.ts:837
res.setHeader("Content-Security-Policy-Report-Only", cspReportOnly);
```

O que é. A política é bem construída — hashes por script inline, `script-src-attr 'none'`, `object-src 'none'`, `base-uri 'self'`, `frame-ancestors 'none'`, sem `unsafe-inline` em `script-src` — e há verificação pós-build (`scripts/verify-csp.ts`). Mas, publicada como **Report-Only**, o navegador apenas relata: **nenhuma injeção é impedida**. O código comenta que a escolha é deliberada para esta etapa (`server.ts:813-814`).

Por que é explorável. Não é uma falha explorável por si: é a ausência da camada que conteria um XSS caso um apareça. Hoje o frontend não tem nenhum vetor de injeção (ver Pontos Fortes), então o risco atual é de regressão futura, não de exploração imediata.

Impacto. Um `dangerouslySetInnerHTML` introduzido num commit futuro, ou um script de terceiro comprometido, executaria sem obstáculo. Perde-se também a proteção contra roubo de token do Firebase via exfiltração para host externo.

Correção. Analisar os relatos já coletados em `cspReports`, confirmar ausência de violações legítimas e promover o header para **Content-Security-Policy**, mantendo o `report-uri` ativo. A infraestrutura de geração e verificação já está pronta.

3. Verificação de posse / integridade de pagamento

BAIXA

A7 · Webhook do Stripe marca pedido como PAGO sem conferir valor nem estado anterior

`server.ts:249-263`

```
const obj = event.data.object as { metadata?: { orderId?: string }; amount?: number };
const orderId = obj.metadata?.orderId;
if (event.type === "payment_intent.succeeded" && orderId) {
  // ... nenhuma comparação entre obj.amount e order.total
  // ... nenhuma checagem do status atual do pedido
  await getAdminDb().collection("orders").doc(orderId).update({
    status: "PAID",
    paidAt: new Date(),
  });
};
```

O que é. A assinatura do evento é verificada corretamente (`server.ts:244`), mas a gravação confia no `orderId` do `metadata` e marca **PAID** sem comparar o valor recebido com o total do pedido e sem consultar o estado atual. O fluxo do Mercado Pago faz as duas coisas — reconsulta o pagamento na API do provedor, compara os centavos e registra **amount_mismatch** (`server/mercadopago/_webhookService.ts:48,99-111`) — a assimetria entre os dois provedores é o achado.

Por que é explorável. Não é explorável remotamente hoje: exige um evento com assinatura válida do Stripe, e o `PaymentIntent` é criado com o total lido do pedido no servidor (`server.ts:196-224`). O risco é de integridade — captura parcial, reembolso parcial ou mudança futura na criação do intent passariam despercebidos. Vale registrar também que **não existe função serverless correspondente em api/**: em produção na Vercel esse webhook não está publicado, então o caminho hoje é o self-host.

Impacto. Pedido marcado como pago por valor inferior ao devido, sem trilha de auditoria da divergência, e reprocessamento silencioso de eventos repetidos.

Correção. Espelhar a lógica do Mercado Pago: comparar `obj.amount` com `order.total` em centavos, recusar e registrar a divergência, aplicar transição de estado idempotente dentro de transação e gravar o evento em `paymentEvents`. Se o Stripe permanecer ativo em produção, publicar a função serverless equivalente.

4. Configuração de CI/CD

INFO

A8 · Workflow ci.yml não declara permissions do GITHUB_TOKEN

[.github/workflows/ci.yml:1-9](#)

```
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  # ← nenhum bloco `permissions:` — herda o padrão do repositório
```

O que é. O workflow **quality.yml** declara corretamente **permissions: contents: read**

([.github/workflows/quality.yml:9-10](#)). O **ci.yml** não declara nada e herda o padrão do repositório, que em repositórios mais antigos é leitura e escrita.

Por que é explorável. Não há execução de código de terceiros no workflow além das dependências instaladas por **npm ci**, e não há segredos expostos ao job. A exposição depende de comprometimento de uma dependência de build.

Impacto. Um script de instalação malicioso numa dependência teria, no pior caso, um token com permissão de escrita no repositório.

Correção. Acrescentar **permissions: contents: read** no topo do **ci.yml**, igual ao **quality.yml**. Os dois workflows executam checagens praticamente idênticas — vale avaliar a consolidação num só.

Recomendações priorizadas

P1 — interrompe exposição ativa de dado ou abuso trivial; fazer primeiro. **P2** — fecha uma superfície explorável sob condição; fazer na sequência. **P3** — endurecimento e defesa em profundidade; planejar.

PRIOR.	AÇÃO	COMO	REF.
P1	Fechar a leitura pública dos anexos de orçamento	storage.rules:33-36 — trocar allow read: if true por isAdmin() (ou restrição pelo UID do caminho). É a única correção que interrompe um vazamento de dado de cliente ativo hoje, e é uma linha.	A1
P1	Aplicar rate limit no endpoint anônimo de produção	api/report-error.ts — portar o limitador que já existe no Express e em api/csp-report.ts , somado a deduplicação por fingerprint. Sem isso, qualquer pessoa escreve no Firestore em laço.	A2
P2	Revalidar host a cada redirect e bloquear faixas internas	server.ts:761 e server/_modelMetadata.ts:284 — redirect: "manual" com reaplicação da allowlist por salto, teto de saltos e recusa de IPs privados, loopback e link-local após resolução DNS.	A3
P2	Mover os contadores de rate limit para armazenamento compartilhado	Vercel KV, Upstash Redis ou documento do Firestore com FieldValue.increment . Destruía a eficácia real de todos os limitadores, inclusive o de P1.	A4
P2	Amarrar nome e foto da avaliação à identidade autenticada	firestore.rules:127-136 — exigir que userName case com o token, ou resolver os campos de exibição na leitura a partir de users/{userId} ; restringir o host de userPhoto .	A5
P3	Promover a CSP de Report-Only para bloqueio	Analisar o que foi coletado em cspReports , validar os fluxos de login, checkout e admin sob bloqueio e publicar como Content-Security-Policy . Toda a infraestrutura de geração e verificação já existe.	A6
P3	Equiparar o webhook do Stripe ao rigor do Mercado Pago	Conferência de valor em centavos, idempotência transacional e registro em paymentEvents — ou decisão explícita de descontinuar o caminho Stripe, já que ele não está publicado na Vercel.	A7
P3	Declarar permissions no ci.yml	Acrescentar permissions: contents: read , como já faz o quality.yml , e avaliar a consolidação dos dois workflows.	A8

Issues para o GitHub

Texto pronto para copiar e colar. São **7 issues** para **8 achados** — os itens de endurecimento relacionados foram agrupados para não inflar o board. Cada bloco começa em **--- ISSUE n ---** e termina em **--- FIM ISSUE n ---**; copie tudo entre os dois delimitadores.

--- ISSUE 1 ---

```
## [Segurança] Anexos de orçamento no Storage são legíveis e enumeráveis por qualquer visitante

**Labels:** `security, severity:high, storage-rules`
**Severidade:** Alta

### Problema

O documento do orçamento no Firestore é corretamente restrito ao dono ou ao admin
(**firestore.rules:361-365**), mas o **anexo correspondente no Storage é público**. O upload
grava em **quotes/{uid}/{timestamp}-{nome}.{ext}** (**src/lib/quotes.ts:223**), então o caminho
carrega o UID do cliente e o nome original do arquivo.

**Por que é explorável**

Nas Storage Rules, **read** concede **get** e **list**. Com leitura liberada, um visitante não
autenticado pode **enumerar o prefixo quotes/ com listAll()** e baixar todas as imagens — sem
precisar adivinhar URL. Não depende de feature flag nem de configuração insegura: é o estado
publicado da regra.

### Evidência

`storage.rules:33-36`

```ts
// Imagens opcionais anexadas a orçamentos salvos pela calculadora.
match /quotes/{allPaths=} {
 allow read: if true; // ← qualquer pessoa, sem login
 allow write: if isAdmin() && isImage() && request.resource.size < 10 * 1024 * 1024;
}
```

### Impacto

Vazamento do portfólio de peças e projetos dos clientes, dos nomes de arquivo enviados e da
correlação UID → peças. Para uma operação B2B de impressão 3D, isso expõe trabalho de terceiros
sob sigilo comercial.

### Sugestão de correção

Trocar **allow read: if true** por **allow read: if isAdmin()** nesse bloco. Se a imagem
precisar ser exibida ao cliente dono, restringir ao UID do próprio caminho (**request.auth.uid
== uid** capturando o segmento) em vez de liberar o prefixo inteiro. As demais pastas (products,
showcase, categories, printers, company) são conteúdo de vitrine e podem seguir públicas.

### Critérios de aceite

- [ ] `storage.rules` deixa de conter `allow read: if true` no bloco `quotes/`
- [ ] Requisição anônima de `listAll()` sobre `quotes/` retorna erro de permissão
- [ ] Download anônimo de uma URL conhecida de `quotes/` retorna 403
- [ ] O admin continua abrindo a imagem no painel de orçamentos sem regressão
- [ ] Teste de regra do Storage cobrindo negação anônima e permissão do admin
- [ ] Correção coberta por teste automatizado que falha sem o patch

_Origem: auditoria de segurança de 1 de setembro de 2026 · commit e099126._
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
## [Segurança] Endpoint anônimo /api/report-error grava no Firestore sem rate limit em produção

**Labels:** `security, severity:medium, backend`
**Severidade:** Média
```

Problema

A rota é anônima por desenho (erro acontece para visitante deslogado) e escreve no Firestore via Admin SDK, além de disparar mensagem no Telegram. A versão Express da mesma rota é protegida por `**rateLimit(20)**` (`**server.ts:671**`), mas `**a função serverless – que é o runtime de produção na Vercel – não tem limitador algum**`.

`**Por que é explorável**`

Basta um laço de POSTs com corpo `**{"message":"x"}**`. Nenhum token, nenhum header especial. Cada requisição cria um documento novo em `**errorReports**` (via `**add()**`, sem deduplicação) e envia uma mensagem ao canal de operação.

Evidência

```
`api/report-error.ts:20-46`

````ts
export default async function handler(req: VercelRequest, res: VercelResponse) {
 if (req.method !== "POST") { /* ... */ }
 try {
 const { valid, data, telegramText } = buildErrorReport(req.body || {});
 if (!valid) { /* ... */ }
 let id: string | null = null;
 if (isAdminSdkConfigured()) {
 const ref = await getAdminDb().collection("errorReports").add(data); // grava sem auth
 id = ref.id;
 }
 await notifyTelegram(telegramText + ...); // dispara no canal de operacao
 }
}
```

### ### Impacto

Inflação ilimitada da coleção `**errorReports**` (custo de escrita e armazenamento no Firestore), inundação do canal do Telegram até o bot ser limitado pela API, e afogamento de relatos legítimos – o painel de erros do admin fica inutilizável.

### ### Sugestão de correção

Portar o limitador para a função serverless – o mesmo padrão já usado em `**api/csp-report.ts:34-50**` – e, para valer sob escala horizontal, ancorá-lo em armazenamento compartilhado (ver A4). Combinar com deduplicação por fingerprint da mensagem, como `**_cspReportRecorder.ts**` já faz.

### ### Critérios de aceite

- [ ] ``api/report-error.ts`` aplica limitação por IP antes de gravar no Firestore
- [ ] Rajada acima do teto responde 429 com header ``Retry-After``
- [ ] Relatos repetidos idênticos são agregados por contador, não duplicados como documentos novos
- [ ] Teste automatizado cobrindo o retorno 429
- [ ] Correção coberta por teste automatizado que falha sem o patch

\_Origem: auditoria de segurança de 1 de setembro de 2026 · commit e099126.\_

## --- FIM ISSUE 2 ---

## --- ISSUE 3 ---

## [Segurança] SSRF: allowlist de host não é revalidada após redirect em proxy-image e model-metadata

`**Labels:**` ``security, severity:medium, backend``  
`**Severidade:**` Média

### ### Problema

Os dois pontos checam o host `**antes**` do fetch e depois seguem redirects automaticamente. O destino final nunca volta à allowlist. A lista padrão (`**server.ts:48-54**`) inclui sites de conteúdo gerado por usuário – thingiverse.com, printables.com, cults3d.com, myminifactory.com – e o casamento aceita subdomínios (`**host.endsWith(".") + allowed**`).

`**Por que é explorável**`

Condicional a um **open redirect** em qualquer host da **allowlist** – classe comum em sites com perfis e links de usuário. Com um, o atacante alcança endereços internos (**169.254.169.254**, **10.0.0.0/8**, **localhost**). Em **/api/proxy-image** o corpo da resposta volta ao chamador quando o **Content-Type** começa com **image**, cabeçalho que o servidor de destino controla. Ambas as rotas são **anônimas**.

### ### Evidência

```
`server.ts:752-762 · server/_modelMetadata.ts:260-285`

```ts
// server.ts – /api/proxy-image (sem autenticação)
if (!isAllowedImageHost(parsed.hostname)) {           // 752 – valida a URL inicial
  res.status(403).json({ error: "host não permitido" });
  return;
}
const upstream = await fetch(rawUrl, {
  headers: { "User-Agent": "..."},
  redirect: "follow",                                // 761 – destino final não é revalidado
});

// server/_modelMetadata.ts – readModelMetadata (sem autenticação)
if (!isAllowedImportHost(targetUrl.hostname)) { /* 260 */ }
const response = await fetch(targetUrl, { headers: {...}, redirect: "follow" }); // 284
```
```

### ### Impacto

Leitura de serviços internos e de endpoints de metadados da nuvem a partir do IP do servidor, com exfiltração do conteúdo pela resposta do proxy. Em **/api/model-metadata**, também uso do servidor como proxy de leitura anônimo.

### ### Sugestão de correção

Usar **redirect: "manual"** e reaplicar **isAllowedImageHost**/**isAllowedImportHost** a cada salto, com teto de saltos. Antes de conectar, resolver o hostname e rejeitar IPs privados, loopback e link-local. Restringir a **allowlist** aos domínios efetivamente necessários e exigir autenticação onde o recurso não é público.

### ### Critérios de aceite

- [ ] Requisição a host permitido que redireciona para IP privado é rejeitada com 403
- [ ] Redirect para host fora da **allowlist** é rejeitado, não seguido
- [ ] Número máximo de saltos aplicado e testado
- [ ] Resolução DNS bloqueia faixas privadas, loopback e link-local antes da conexão
- [ ] Testes cobrindo redirect para host proibido e para IP interno
- [ ] Correção coberta por teste automatizado que falha sem o patch

\_Origem: auditoria de segurança de 1 de setembro de 2026 · commit e099126.\_

## --- FIM ISSUE 3 ---

## --- ISSUE 4 ---

## [Segurança] Rate limiters em memória são inefetivos no runtime serverless

**Labels:** `security, severity:low, infra`  
**Severidade:** Baixa

### ### Problema

Os contadores ficam num **Map** local. Em Express de instância única isso funciona; na Vercel, cada invocação pode cair numa instância diferente e instâncias frias começam com o mapa vazio.

**Por que é explorável**

Não exige truque: o próprio modelo de escala do provedor dilui o limite. O teto efetivo vira **máximo por minuto × número de instâncias ativas**, valor que o atacante aumenta simplesmente paralelizando as requisições.

### ### Evidência

```
`server.ts:65-83 · api/csp-report.ts:21 · api/mercadopago/payment-status.ts:8 ·
api/mercadopago/process-payment.ts:11`
```

```

```ts
// server.ts:65 – e o mesmo padrão repetido nas funções serverless
const rateBuckets = new Map<string, { count: number; resetAt: number }>();
function rateLimit(maxPerMinute: number) {
  return (req, res, next) => {
    const key = `${req.path}:${req.ip}`;
    // ... contador vive apenas nesta instância do processo
  }
}
```

Impacto

Enfraquece a proteção de **/api/mercadopago/process-payment** (chamadas ao provedor de pagamento), **/api/csp-report** e **/api/calculator/extract-slicer** (cota da API Gemini). É o multiplicador que agrava A2.

Sugestão de correção

Mover a contagem para armazenamento compartilhado (Vercel KV, Upstash Redis ou um documento do Firestore com **FieldValue.increment**) ou adotar o rate limiting da borda do provedor. Manter o limitador em memória apenas como segunda linha.

Critérios de aceite

- [] Contadores persistem fora da memória do processo
- [] Teto respeitado com requisições paralelas atingindo instâncias diferentes
- [] Rotas de pagamento, CSP e extract-slicer cobertas
- [] Teste de carga confirmando 429 no teto configurado
- [] Correção coberta por teste automatizado que falha sem o patch

Origem: auditoria de segurança de 1 de setembro de 2026 · commit e099126.

```

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

```

[Segurança] Avaliações permitem personificação: userName e userPhoto não são amarrados ao usuário

Labels: `security, severity:low, firestore-rules`
Severidade: Baixa

Problema

A regra amarra corretamente **userId** e força um ID determinístico (uma avaliação por usuário por produto), mas **userName** e **userPhoto** – os campos efetivamente exibidos – aceitam qualquer string. O cliente legítimo envia os dados do próprio perfil (**src/hooks/useReviews.ts:91-92**), porém isso é convenção do frontend, não regra.

Por que é explorável

Qualquer usuário autenticado escreve o documento direto pelo SDK web com **userName: "INOVAPRO3D Oficial"* e um **userPhoto** apontando para qualquer URL. A avaliação é renderizada publicamente em **src/components/product/ProductReviews.tsx:211,223**.

Evidência

`firestore.rules:127-136, 237-241`

```ts
function isValidReview(data) {
  return data.productId is string && data.productId.size() <= 128 &&
    data.userId is string &&
    data.rating is number && data.rating >= 1 && data.rating <= 5 &&
    optionalString(data.get('comment', null), 1000) &&
    optionalString(data.get('userName', null), 160) && // ← conteúdo livre
    optionalString(data.get('userPhoto', null), 2048) && // ← URL livre
  /* ... */
}
allow create: if isSignedIn()
  && isValidReview(incoming())
  && incoming().userId == request.auth.uid // ← userId é amarrado
  && reviewId == incoming().productId + '_' + request.auth.uid
  && !('hidden' in incoming());

```



```
...
```

Impacto

Personificação da marca ou de outro cliente em avaliações públicas de produto, com peso de prova social. A URL da foto também vira um beacon: carrega do navegador de todo visitante da página do produto, revelando IP e User-Agent a um terceiro escolhido pelo autor da avaliação.

Sugestão de correção

Ou remover `**userName**`/`**userPhoto**` do documento e resolvê-los na leitura a partir de `**users/{userId}**`, ou exigir na regra que casem com o token (`**incoming().userName == request.auth.token.name**`). Para a foto, restringir o host a domínios conhecidos de avatar.

Critérios de aceite

- [] Avaliação com ``userName`` divergente da identidade autenticada é rejeitada pela regra
- [] ``userPhoto`` fora dos hosts permitidos é rejeitada
- [] Avaliações existentes continuam renderizando (migração ou leitura por junção)
- [] Teste de regra cobrindo tentativa de personificação
- [] Correção coberta por teste automatizado que falha sem o patch

Origem: auditoria de segurança de 1 de setembro de 2026 · commit e099126.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Promover CSP de Report-Only para bloqueio e alinhar permissions do CI

Labels: `security, severity:low, hardening`
Severidade: Baixa · **Achados:** A6, A8

Problema

A6 – CSP publicada apenas em Report-Only: a política existe mas não bloqueia

A política é bem construída – hashes por script inline, `**script-src-attr 'none'**`, `**object-src 'none'**`, `**base-uri 'self'**`, `**frame-ancestors 'none'**`, sem `**unsafe-inline**` em `**script-src**` – e há verificação pós-build (`**scripts/verify-csp.ts**`). Mas, publicada como `**Report-Only**`, o navegador apenas relata: `**nenhuma injeção é impedida**`. O código comenta que a escolha é deliberada para esta etapa (`**server.ts:813-814**`).

Por que é explorável

Não é uma falha explorável por si: é a ausência da camada que conteria um XSS caso um apareça. Hoje o frontend não tem nenhum vetor de injeção (ver Pontos Fortes), então o risco atual é de regressão futura, não de exploração imediata.

A8 – Workflow ci.yml não declara permissions do GITHUB_TOKEN

O workflow `**quality.yml**` declara corretamente `**permissions: contents: read**` (`**github/workflows/quality.yml:9-10**`). O `**ci.yml**` não declara nada e herda o padrão do repositório, que em repositórios mais antigos é leitura `_e escrita_`.

Por que é explorável

Não há execução de código de terceiros no workflow além das dependências instaladas por `**npm ci**`, e não há segredos expostos ao job. A exposição depende de comprometimento de uma dependência de build.

Evidência

`vercel.json:40-41 · server.ts:837`

```
```ts
```

```
// vercel.json:40
```

```
{ "key": "Content-Security-Policy-Report-Only", "value": "default-src 'self'; script-src 'self' 'sha256-...' ..." }
```

```
// server.ts:837
```

```
res.setHeader("Content-Security-Policy-Report-Only", cspReportOnly);
```

```
...
```

```

`.github/workflows/ci.yml:1-9`

```ts
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
    # ← nenhum bloco `permissions:` – herda o padrão do repositório
    ...

### Impacto

A6: Um dangerouslySetInnerHTML introduzido num commit futuro, ou um script de terceiro comprometido, executaria sem obstáculo. Perde-se também a proteção contra roubo de token do Firebase via exfiltração para host externo.

A8: Um script de instalação malicioso numa dependência teria, no pior caso, um token com permissão de escrita no repositório.

### Sugestão de correção

A6: Analisar os relatos já coletados em cspReports, confirmar ausência de violações legítimas e promover o header para Content-Security-Policy, mantendo o report-uri ativo. A infraestrutura de geração e verificação já está pronta.

A8: Acrescentar permissions: contents: read no topo do ci.yml, igual ao quality.yml. Os dois workflows executam checagens praticamente idênticas – vale avaliar a consolidação num só.

### Critérios de aceite

- [ ] Relatos de cspReports revisados e sem violação legítima pendente
- [ ] Header publicado como Content-Security-Policy em vercel.json e em server.ts
- [ ] Fluxos de login, checkout Stripe/Mercado Pago e painel admin validados sob bloqueio
- [ ] report-uri mantido para monitorar regressões
- [ ] ci.yml declara permissions: contents: read
- [ ] Padrão do repositório revisado em Settings → Actions → Workflow permissions
- [ ] Duplicação entre ci.yml e quality.yml avaliada
- [ ] Correção coberta por teste automatizado que falha sem o patch

_Origem: auditoria de segurança de 1 de setembro de 2026 · commit e099126._

```

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

```

## [Segurança] Webhook do Stripe marca pedido como pago sem conferir valor nem idempotência

Labels: `security, severity:low, payments`
Severidade: Baixa

### Problema

A assinatura do evento é verificada corretamente (server.ts:244), mas a gravação confia no orderId do metadata e marca PAID sem comparar o valor recebido com o total do pedido e sem consultar o estado atual. O fluxo do Mercado Pago faz as duas coisas – reconsulta o pagamento na API do provedor, compara os centavos e registra amount_mismatch (server/mercadopago/_webhookService.ts:48,99-111) – a assimetria entre os dois provedores é o achado.

Por que é explorável

Não é explorável remotamente hoje: exige um evento com assinatura válida do Stripe, e o PaymentIntent é criado com o total lido do pedido no servidor (server.ts:196-224). O risco é de integridade – captura parcial, reembolso parcial ou mudança futura na criação do intent passariam despercebidos. Vale registrar também que não existe função serverless correspondente em api/: em produção na Vercel esse webhook não está publicado, então o caminho hoje é o self-host.

### Evidência

```

```
`server.ts:249-263`
```

```
``ts
const obj = event.data.object as { metadata?: { orderId?: string }; amount?: number };
const orderId = obj.metadata?.orderId;
if (event.type === "payment_intent.succeeded" && orderId) {
  // ... nenhuma comparação entre obj.amount e order.total
  // ... nenhuma checagem do status atual do pedido
  await getAdminDb().collection("orders").doc(orderId).update({
    status: "PAID",
    paidAt: new Date(),
  });
}
```

Impacto

Pedido marcado como pago por valor inferior ao devido, sem trilha de auditoria da divergência, e reprocessamento silencioso de eventos repetidos.

Sugestão de correção

Espelhar a lógica do Mercado Pago: comparar **obj.amount** com **order.total** em centavos, recusar e registrar a divergência, aplicar transição de estado idempotente dentro de transação e gravar o evento em **paymentEvents**. Se o Stripe permanecer ativo em produção, publicar a função **serverless** equivalente.

Critérios de aceite

- [] Webhook compara valor recebido com `order.total` em centavos e recusa divergência
- [] Divergência registrada em `paymentEvents` com valores esperado e recebido
- [] Evento repetido não reprocessa o pedido (idempotência verificada em transação)
- [] Decisão registrada sobre publicar ou remover o caminho Stripe em produção
- [] Correção coberta por teste automatizado que falha sem o patch

Origem: auditoria de segurança de 1 de setembro de 2026 · commit e099126.

--- FIM ISSUE 7 ---